基于零售交易数据的Spark数据处理与分析

https://dblab.xmu.edu.cn/blog/2738/

本案例数据集是来自Kaggle的一个跨国在线零售业务的交易数据，采用Python为编程语言，采用Hadoop存储数据，采用Spark对数据进行处理分析，并使用Echarts做数据可视化。

一、环境搭建

本次作业使用的环境和软件如下：

（1）Linux操作系统：Ubuntu 16.04
（2）Python：3.5.2
（3）Hadoop：3.1.3 （查看安装教程）
（4）Spark：2.4.0 （查看安装教程）
（5）Bottle：v0.12.18

Bottle是一个快速、简洁、轻量级的基于WSIG的微型Web框架，此框架除了Python的标准库外，不依赖任何其他模块。安装方法是，打开Linux终端，执行如下命令：

```
sudo apt-get install python3-pip
pip3 install bottle
```

Shell 命令

至此，环境搭建过程结束。

二、数据预处理

本次作业使用的数据集是来自Kaggle的一个跨国在线零售业务的交易数据（从百度网盘下载数据集，提取码：pyej），该公司在英国注

册，主要销售礼品。数据集E_Commerce_Data.csv包含541909个记录，时间跨度为2010-12-01到2011-12-09，每个记录由8个属性组成，具体的含义如下表：

字段名称 类型 含义 举例

InvoiceNo string 订单编号（退货订单以C开头） 536365
StockCode string 产品代码 85123A
Description string 产品描述 WHITE METAL LANTERN
Quantity integer 购买数量（负数表示退货） 6
InvoiceDate string 订单日期和时间 12/1/2010 8:26
UnitPrice double 单价（英镑） 3.39
CustomerID integer 客户编号 17850
Country string 国家名称 United Kingdom

首先，将数据集E_Commerce_Data.csv上传至hdfs上，命令如下：

```
hdfs dfs -put E_Commerce_Data.csv
```

Shell 命令

接着，使用如下命令进入pyspark的交互式编程环境，对数据进行初步探索和清洗：

```
cd /usr/local/spark #进入Spark安装目录
./bin/pyspark
```

Shell 命令

（1）读取在HDFS上的文件，以csv的格式读取，得到DataFrame对象。

```
>>>
df=spark.read.format('com.databricks.spark.csv
').options(header='true',
inferschema='true').load('E_Commerce_Data.csv'
)
```

Python

（2）查看数据集的大小，输出541909，不包含标题行

```
>>> df.count()
```

Python

（3）打印数据集的schema，查看字段及其类型信息。输出内容就是上文中的属性表。

```
>>> df.printSchema()
```
Python

（4）创建临时视图data。

```
>>> df.createOrReplaceTempView("data")
```
Python

（5）由于顾客编号CustomID和商品描述Description均存在部分缺失，所以进行数据清洗，过滤掉有缺失值的记录。特别地，由于CustomID为integer类型，所以该字段若为空，则在读取时被解析为0，故用df["CustomerID"]!=0 条件过滤。

```
>>> clean=df.filter(df["CustomerID"]!
=0).filter(df["Description"]!="")
```
Python

（6）查看清洗后的数据集的大小，输出406829。

```
>>> clean.count()
```
Python

（7）数据清洗结束。根据作业要求，预处理后需要将数据写入HDFS。将清洗后的文件以csv的格式，写入E_Commerce_Data_Clean.csv中（实际上这是目录名，真正的文件在该目录下，文件名类似于part-00000），需要确保HDFS中不存在这个目录，否则写入时会报“already exists”错误。

```
>>>
clean.write.format("com.databricks.spark.csv")
.options(header='true',inferschema='true').sav
e('E_Commerce_Data_Clean.csv')
```
Python

至此，数据预处理完成。接下来将使用python编写应用程序，对清洗后的数据集进行统计分析。

三、数据分析

备注：全部源代码文件可以从百度网盘下载：（从百度网盘下载源代码，提取码：pyej）
创建project.py文件，用于编写应用程序。
首先，导入需要用到的python模块。

```python
# -*- coding: utf-8 -*-
from pyspark import SparkContext
from pyspark.sql import SparkSession
from pyspark.sql.types import StringType, DoubleType, IntegerType, StructField, StructType
import json
import os
```

接着，获取spark sql的上下文。

```python
sc = SparkContext('local', 'spark_project')
sc.setLogLevel('WARN')
spark = SparkSession.builder.getOrCreate()
```

最后，从HDFS中以csv的格式读取清洗后的数据目录E_Commerce_Data_Clean.csv，程序会取出该目录下的所有数据文件，得到DataFrame对象，并创建临时视图data用于后续分析。

```python
df = spark.read.format('com.databricks.spark.csv').options(header='true', inferschema='true').load('E_Commerce_Data_Clean.csv')
df.createOrReplaceTempView("data")
```

为方便统计结果的可视化，将结果导出为json文件供web页面渲染。
使用save方法导出数据：

```python
def save(path, data):
    with open(path, 'w') as f:
        f.write(data)
```

准备工作完成。接下来对数据进行分析，分为概览和关系两个部分。

1.概览

（1）客户数最多的10个国家

每个客户由编号CustomerID唯一标识，所以客户的数量为COUNT(DISTINCT CustomerID)，再按照国家Country分组统计，根据客户数降序排序，筛选出10个客户数最多的国家。得到的countryCustomerDF为DataFrame类型，执行collect()方法即可将结果以数组的格式返回。

```python
def countryCustomer():
    countryCustomerDF = spark.sql("SELECT Country,COUNT(DISTINCT CustomerID) AS countOfCustomer FROM data GROUP BY Country ORDER BY countOfCustomer DESC LIMIT 10")
    return countryCustomerDF.collect()
```

Python

最后调用save方法就可以将结果导出至文件了，格式如下：

```
[国家名称,客户数]
```

（2）销量最高的10个国家

Quantity字段表示销量，因为退货的记录中此字段为负数，所以使用SUM(Quantity)即可统计出总销量，即使有退货的情况。再按照国家Country分组统计，根据销量降序排序，筛选出10个销量最高的国家。得到的countryQuantityDF为DataFrame类型，执行collect()方法即可将结果以数组的格式返回。

```python
def countryQuantity():
    countryQuantityDF = spark.sql("SELECT Country,SUM(Quantity) AS sumOfQuantity FROM data GROUP BY Country ORDER BY sumOfQuantity DESC LIMIT 10")
    return countryQuantityDF.collect()
```

Python
最后调用save方法就可以将结果导出至文件了，格式如下：

[国家名称,销量]

（3）各个国家的总销售额分布情况

UnitPrice 字段表示单价，Quantity字段表示销量，退货的记录中Quantity字段为负数，所以使用SUM(UnitPrice*Quantity)即可统计出总销售额，即使有退货的情况。再按照国家Country分组统计，计算出各个国家的总销售额。得到的countrySumOfPriceDF为DataFrame类型，执行collect()方法即可将结果以数组的格式返回。

```
def countrySumOfPrice():
    countrySumOfPriceDF = spark.sql("SELECT Country,SUM(UnitPrice*Quantity) AS sumOfPrice FROM data GROUP BY Country")
    return countrySumOfPriceDF.collect()
```

Python
最后调用save方法就可以将结果导出至文件了，格式如下：

[国家名称,总销售额]

（4）销量最高的10个商品

Quantity字段表示销量，退货的记录中Quantity字段为负数，所以使用SUM(Quantity)即可统计出总销量，即使有退货的情况。再按照商品编码StockCode分组统计，计算出各个商品的销量。得到的stockQuantityDF为DataFrame类型，执行collect()方法即可将结果以数组的格式返回。

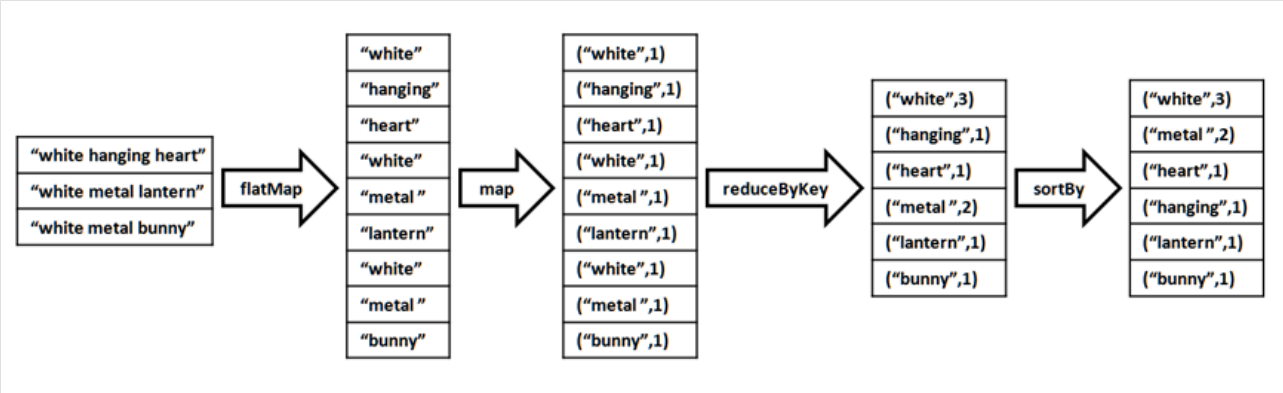
```
def stockQuantity():
    stockQuantityDF = spark.sql("SELECT StockCode,SUM(Quantity) AS sumOfQuantity FROM data GROUP BY StockCode ORDER BY sumOfQuantity DESC LIMIT 10")
    return stockQuantityDF.collect()
```

Python
最后调用save方法就可以将结果导出至文件了，格式如下：

[商品编号,销量]

（5）商品描述的热门关键词Top300

Description字段表示商品描述，由若干个单词组成，使用LOWER(Description)将单词统一转换为小写。此时的结果为DataFrame类型，转化为rdd后进行词频统计，再根据单词出现的次数进行降序排序，流程图如下：



得到的结果为RDD类型，为其制作表头wordCountSchema，包含word和count属性，分别为string类型和integer类型。调用createDataFrame()方法将其转换为DataFrame类型的wordCountDF，将word为空字符串的记录剔除掉，调用take()方法得到出现次数最多的300个关键词，以数组的格式返回。

```
def wordCount():
    wordCount = spark.sql("SELECT
LOWER(Description) as description from
data").rdd.flatMap(lambda
line:line['description'].split("
")).map(lambda word:
(word,1)).reduceByKey(lambda
a,b:a+b).repartition(1).sortBy(lambda
x:x[1],False)

    wordCountSchema =
StructType([StructField("word", StringType(),
True),StructField("count", IntegerType(),
True)])
```

```
    wordCountDF =
spark.createDataFrame(wordCount,
wordCountSchema)
    wordCountDF =
wordCountDF.filter(wordCountDF["word"]!='')
    return wordCountDF.take(300)
```
Python

最后调用save方法就可以将结果导出至文件了，格式如下：

```
[关键词,次数]
```

（6）退货订单数最多的10个国家

InvoiceNo字段表示订单编号，所以订单总数为COUNT(DISTINCT InvoiceNo)，由于退货订单的编号的首个字母为C，例如C540250，所以利用WHERE InvoiceNo LIKE 'C%'子句即可筛选出退货的订单，再按照国家Country分组统计，根据退货订单数降序排序，筛选出10个退货订单数最多的国家。得到的countryReturnInvoiceDF为DataFrame类型，执行collect()方法即可将结果以数组的格式返回。

```
def countryReturnInvoice():
    countryReturnInvoiceDF = spark.sql("SELECT
Country,COUNT(DISTINCT InvoiceNo) AS
countOfReturnInvoice FROM data WHERE InvoiceNo
LIKE 'C%' GROUP BY Country ORDER BY
countOfReturnInvoice DESC LIMIT 10")
    return countryReturnInvoiceDF.collect()
```
Python

最后调用save方法就可以将结果导出至文件了，格式如下：

```
[国家名称,退货订单数]
```

2.关系

（7）月销售额随时间的变化趋势

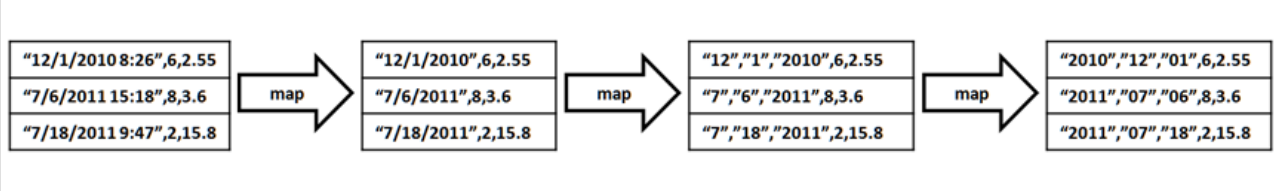
统计月销售额需要3个字段的信息，分别为订单日期InvoiceDate，销量Quantity和单价UnitPrice。由于InvoiceDate字段格式不容易处

理，例如“8/5/2011 16:19”，所以需要对这个字段进行格式化操作。由于统计不涉及小时和分钟数，所以只截取年月日部分，并且当数值小于10时补前置0来统一格式，期望得到年、月、日3个独立字段。先实现formatData()方法，利用rdd对日期、销量和单价字段进行处理。

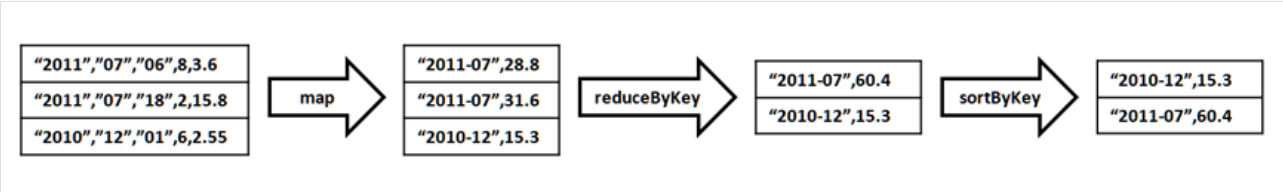
```python
def formatData():
    tradeRDD = df.select("InvoiceDate","Quantity","UnitPrice",).rdd
    result1 = tradeRDD.map(lambda line: (line['InvoiceDate'].split(" ")[0], line['Quantity'] , line['UnitPrice']))
    result2 = result1.map(lambda line: (line[0].split("/"), line[1], line[2]))
    result3 = result2.map(lambda line: (line[0][2], line[0][0] if len(line[0][0])==2 else "0"+line[0][0], line[0][1] if len(line[0][1])==2 else "0"+line[0][1], line[1], line[2]))
    return result3
```

Python

流程图如下：



由于要统计的是月销售额的变化趋势，所以只需将日期转换为“2011-08”这样的格式即可。而销售额表示为单价乘以销量，需要注意的是，退货时的销量为负数，所以对结果求和可以表示销售额。RDD的转换流程如下：



得到的结果为RDD类型，为其制作表头schema，包含date和tradePrice属性，分别为string类型和double类型。调用createDataFrame()方法将其转换为DataFrame类型的tradePriceDF，调用collect()方法将结果以数组的格式返回。

```
    def tradePrice():
        result3 = formatData()
        result4 = result3.map(lambda line:
    (line[0]+"-"+line[1],line[3]*line[4]))
        result5 = result4.reduceByKey(lambda
    a,b:a+b).sortByKey()
        schema = StructType([StructField("date",
    StringType(), True),StructField("tradePrice",
    DoubleType(), True)])
        tradePriceDF =
    spark.createDataFrame(result5, schema)
        return tradePriceDF.collect()
```

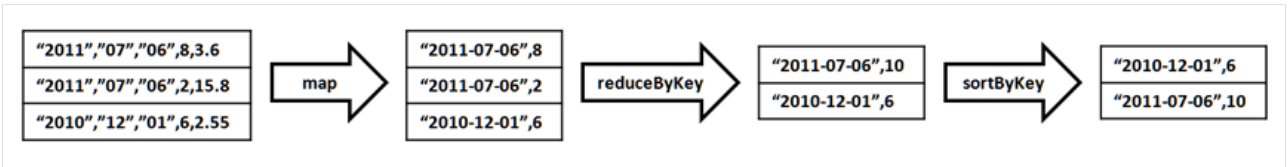
Python

最后调用save方法就可以将结果导出至文件了，格式如下：

```
[日期,销售额]
```

（8）日销量随时间的变化趋势

由于要统计的是日销量的变化趋势，所以只需将日期转换为“2011-08-05”这样的格式即可。先调用上例的formatData()方法对日期格式进行格式化。RDD的转换流程如下：



得到的结果为RDD类型，为其制作表头schema，包含date和saleQuantity属性，分别为string类型和integer类型。调用createDataFrame()方法将其转换为DataFrame类型的saleQuantityDF，调用collect()方法将结果以数组的格式返回。

```
    def saleQuantity():
```

```
    result3 = formatData()
    result4 = result3.map(lambda line:
(line[0]+"-"+line[1]+"-"+line[2],line[3]))
    result5 = result4.reduceByKey(lambda
a,b:a+b).sortByKey()
    schema = StructType([StructField("date",
StringType(),
True),StructField("saleQuantity",
IntegerType(), True)])
    saleQuantityDF =
spark.createDataFrame(result5, schema)
    return saleQuantityDF.collect()
```

Python

最后调用save方法就可以将结果导出至文件了，格式如下：

```
[日期,销量]
```

（9）各国的购买订单量和退货订单量的关系

InvoiceNo字段表示订单编号，退货订单的编号的首个字母为C，例如C540250。利用COUNT(DISTINCT InvoiceNo)子句统计订单总量，再分别用WHERE InvoiceNo LIKE 'C%'和WHERE InvoiceNo NOT LIKE 'C%'统计出退货订单量和购买订单量。接着按照国家Country分组统计，得到的returnDF和buyDF均为DataFrame类型，分别表示退货订单和购买订单，如下所示：

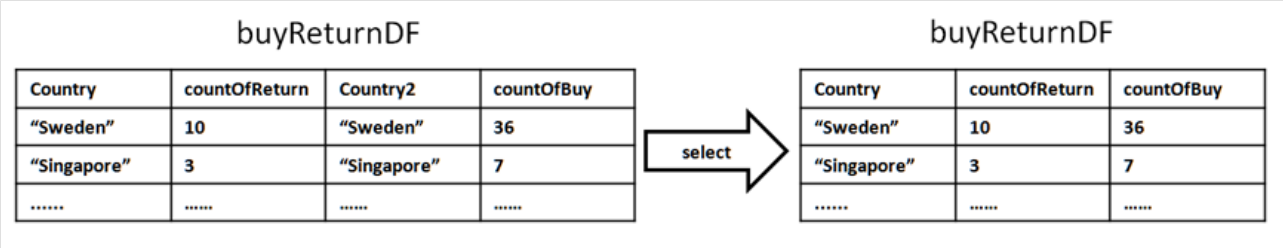
returnDF

Country	countOfReturn
“Sweden”	10
“Singapore”	3
……	……

buyDF

Country2	countOfBuy
“Sweden”	36
“Singapore”	7
……	……

再对这两个DataFrame执行join操作，连接条件为国家Country相同，得到一个DataFrame。但是这个DataFrame中有4个属性，包含2个重复的国家Country属性和1个退货订单量和1个购买订单量，为减少冗余，对结果筛选3个字段形成buyReturnDF。如下所示：



最后执行collect()方法即可将结果以数组的格式返回。

```python
def buyReturn():
    returnDF = spark.sql("SELECT Country AS Country,COUNT(DISTINCT InvoiceNo) AS countOfReturn FROM data WHERE InvoiceNo LIKE 'C%' GROUP BY Country")
    buyDF = spark.sql("SELECT Country AS Country2,COUNT(DISTINCT InvoiceNo) AS countOfBuy FROM data WHERE InvoiceNo NOT LIKE 'C%' GROUP BY Country2")
    buyReturnDF = returnDF.join(buyDF, returnDF["Country"] == buyDF["Country2"], "left_outer")
    buyReturnDF = buyReturnDF.select(buyReturnDF["Country"],buyReturnDF["countOfBuy"],buyReturnDF["countOfReturn"])
    return buyReturnDF.collect()
```

Python

最后调用save方法就可以将结果导出至文件了，格式如下：

```
[国家名称,购买订单数,退货订单数]
```

（10）商品的平均单价与销量的关系

由于商品的单价UnitPrice是不断变化的，所以使用平均单价AVG(DISTINCT UnitPrice)来衡量一个商品。再利用SUM(Quantity)计算出销量，将结果按照商品的编号进行分组统计，执行collect()方法即可将结果以数组的格式返回。

```python
def unitPriceSales():
```

```
    unitPriceSalesDF = spark.sql("SELECT
StockCode,AVG(DISTINCT UnitPrice) AS
avgUnitPrice,SUM(Quantity) AS sumOfQuantity
FROM data GROUP BY StockCode")
    return unitPriceSalesDF.collect()
```
Python

最后调用save方法就可以将结果导出至文件了，格式如下：

```
[商品编号,平均单价,销量]
```

3.小结

在project.py中添加main函数，将上面的分析过程整合起来方便进行调用，代码如下：

```
if __name__ == "__main__":

    base = "static/"
    if not os.path.exists(base):
        os.mkdir(base)

    m = {
        "countryCustomer": {
            "method": countryCustomer,
            "path": "countryCustomer.json"
        },
        "countryQuantity": {
            "method": countryQuantity,
            "path": "countryQuantity.json"
        },
        "countrySumOfPrice": {
            "method": countrySumOfPrice,
            "path": "countrySumOfPrice.json"
        },
        "stockQuantity": {
            "method": stockQuantity,
            "path": "stockQuantity.json"
        },
```

```
        "wordCount": {
            "method": wordCount,
            "path": "wordCount.json"
        },
        "countryReturnInvoice": {
            "method": countryReturnInvoice,
            "path":
"countryReturnInvoice.json"
        },
        "tradePrice": {
            "method": tradePrice,
            "path": "tradePrice.json"
        },
        "saleQuantity": {
            "method": saleQuantity,
            "path": "saleQuantity.json"
        },
        "buyReturn": {
            "method": buyReturn,
            "path": "buyReturn.json"
        },
        "unitPriceSales": {
            "method": unitPriceSales,
            "path": "unitPriceSales.json"
        }
    }

    for k in m:
        p = m[k]
        f = p["method"]
        save(base + m[k]["path"],
json.dumps(f()))
        print ("done -> " + k + " , save to
-> " + base + m[k]["path"])
```
Python

上面的代码将所有的函数整合在变量 m中，通过循环调用上述所有方法并导出json文件到当前路径的static目录下。

最后利用如下指令运行分析程序：

```
cd /usr/local/spark
./bin/spark-submit project.py
```
Shell 命令

四、可视化方法

备注：全部源代码文件可以从百度网盘下载：（从百度网盘下载源代码，提取码：pyej）

本次作业的结果可视化使用百度开源的免费数据展示框架Echarts。Echarts是一个纯Javascript的图表库，可以流畅地运行在PC和移动设备上，兼容当前绝大部分浏览器，底层依赖轻量级的Canvas类库ZRender，提供直观，生动，可交互，可高度个性化定制的数据可视化图表。

编写web.py程序，实现一个简单的web服务器，代码如下：

```
from bottle import route, run, static_file
import json

@route('/static/<filename>')
def server_static(filename):
    return static_file(filename, root="./
static")

@route("/<name:re:.*\.html>")
def server_page(name):
    return static_file(name, root=".")

@route("/")
def index():
    return static_file("index.html",
root=".")

run(host="0.0.0.0", port=9999)
```
Python

bottle服务器对接收到的请求进行路由，规则如下：

（1）访问/static/时，返回静态文件

（2）访问/.html时，返回网页文件

（3）访问/时，返回首页index.html

服务器的9999端口监听来自任意ip的请求（前提是请求方能访问到这台服务器）。

首页index.html的主要代码如下(由于篇幅较大，只截取主要的部分)

```
<!DOCTYPE html>
<html lang="en">

<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width,height=device-height">
    <title>E-Commerce-Data 在线零售业务数据分析</title>
<style>
/* 省略 */
    </style>
</head>

<body>
    <div class="container">
    /* 只展示第一个统计结果的代码，其余省略 */
        <div class="chart-group">
            <h3>(1) 客户数最多的10个国家
            <br>
            <small style="font-size: 72%;">
                ——英国的客户最多，达到3950个，数量远大于其他国家；其次是德国、法国、西班牙等
            </small>
            </h3>
            <iframe src="countryCustomer.html" class="frame" frameborder="0"></iframe>
        </div>
    </div>
```

```
    <script>document.body.clientHeight;</script>
</body>
</html>
```

HTML

图表页通过一个iframe嵌入到首页中。以第一个统计结果的网页countryCustomer.html为例，展示主要代码：

```
<!DOCTYPE html>
<html lang="en">

<head>
    <meta charset="UTF-8">
    <style>
/* 省略 */
    </style>
</head>

<body>
<div id="chart" style="width:95%;height:95%;"></div>
<script src="static/jquery-3.2.1.min.js"></script>
<script src="static/echarts-4.7.0.min.js"></script>
<script>
    var myChart = echarts.init(document.getElementById('chart'));
    myChart.setOption(
    {
        color: ['#3398DB'],
        tooltip: {
            trigger: 'axis',
            axisPointer: {
                type: 'shadow'
            }
        },
```

```
        grid: {
            left: '3%',
            right: '4%',
            bottom: '3%',
            containLabel: true
        },
        xAxis: [
            {
                name: '国家',
                data: [],
                axisTick: {
                    alignWithLabel: true
                },
                axisLabel: {
                   interval:0,
                   rotate:40
                }
            }
        ],
        yAxis: [
            {
                name: '客户数',
            }
        ],
        series: [
            {
                name: '客户数',
                type: 'bar',
                barWidth: '60%',
                data: []
            }
        ]
    });
    myChart.showLoading();
    $.getJSON("/static/countryCustomer.json", data => {
        var names=[];
```

```
        var nums=[];

        data = data.map(v => ({
            country: v[0],
            customer: parseInt(v[1]),
        }))

        for(var i=0;i<data.length;i++){
            names.push(data[i].country);
            nums.push(data[i].customer);
        }
        myChart.setOption({
            xAxis: {
                data: names
            },
            series: [{
                data: nums
            }]
        });
        myChart.hideLoading();
    })
</script>
</body>
</html>
```
HTML

代码完成后，在代码所在的根目录下执行以下指令启动web服务器：

```
python3 web.py
```
Shell 命令

若打印出以下信息则表示web服务启动成功。接着，可以通过使用浏览器访问网页的方式查看统计结果。

```
Bottle v0.12.18 server starting up (using
WSGIRefServer())...
Listening on http://0.0.0.0:9999/
Hit Ctrl-C to quit.
```

为方便运行程序，编写run.sh脚本，内容如下。首先向spark提交project.py程序对数据进行统计分析，生成的json文件会存入当前路

径的static目录下；接着运行web.py程序，即启动web服务器对分析程序生成的json文件进行解析渲染，方便用户通过浏览器查看统计结果的可视化界面。

```
#!/bin/bash
cd /usr/local/spark
./bin/spark-submit project.py
python3 web.py
```

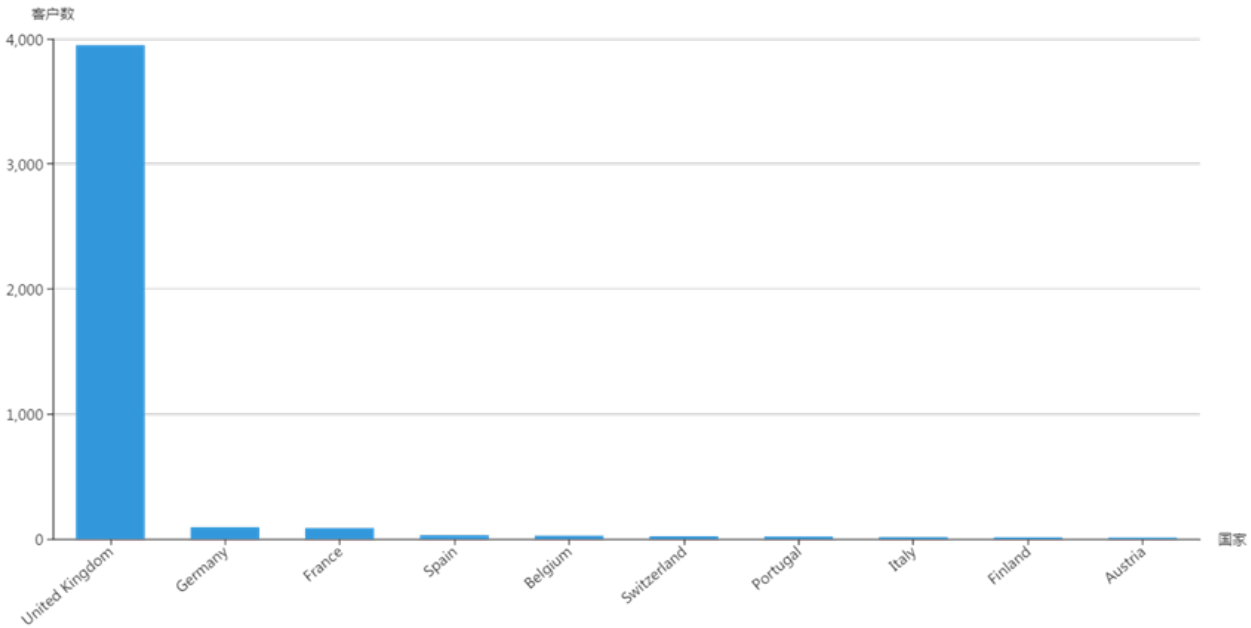
提示：运行程序前，应先按照本作业的步骤二，将清洗后的数据集保存到HDFS分布式文件系统中，目录命名为E_Commerce_Data_Clean.csv，否则project.py程序会由于读取不到其中的数据文件而报错。

五、结果可视化

1.概览

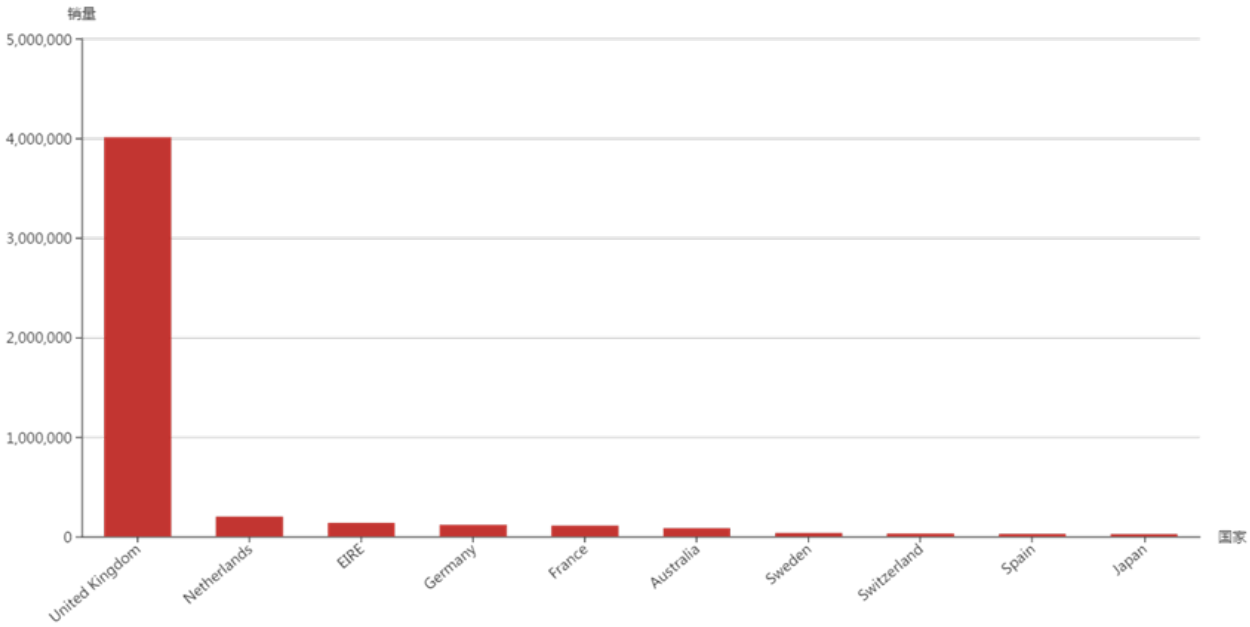
（1）客户数最多的10个国家

——英国的客户最多，达到3950个，数量远大于其他国家；其次是德国、法国、西班牙等。



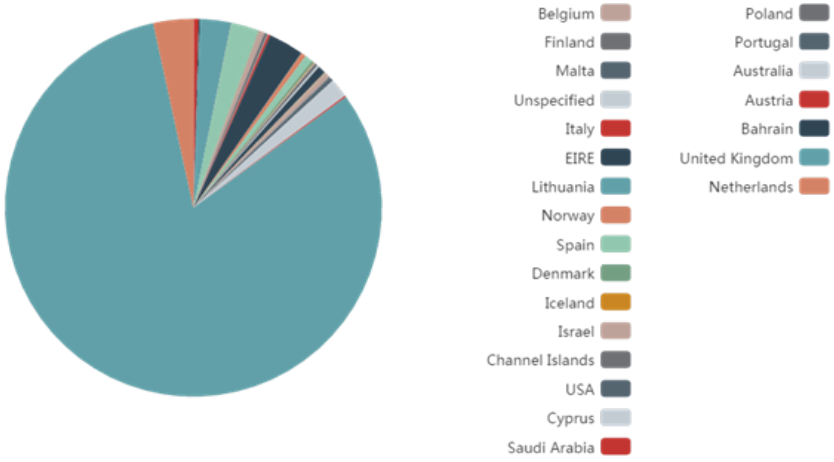
（2）销量最高的10个国家

——英国的销量最高，达到4008533件，远大于其他国家；其次是新西兰、爱尔兰、德国等。



（3）各个国家的总销售额分布情况

——英国的总销售额最高，达到6767873.394英镑，占比81.54%



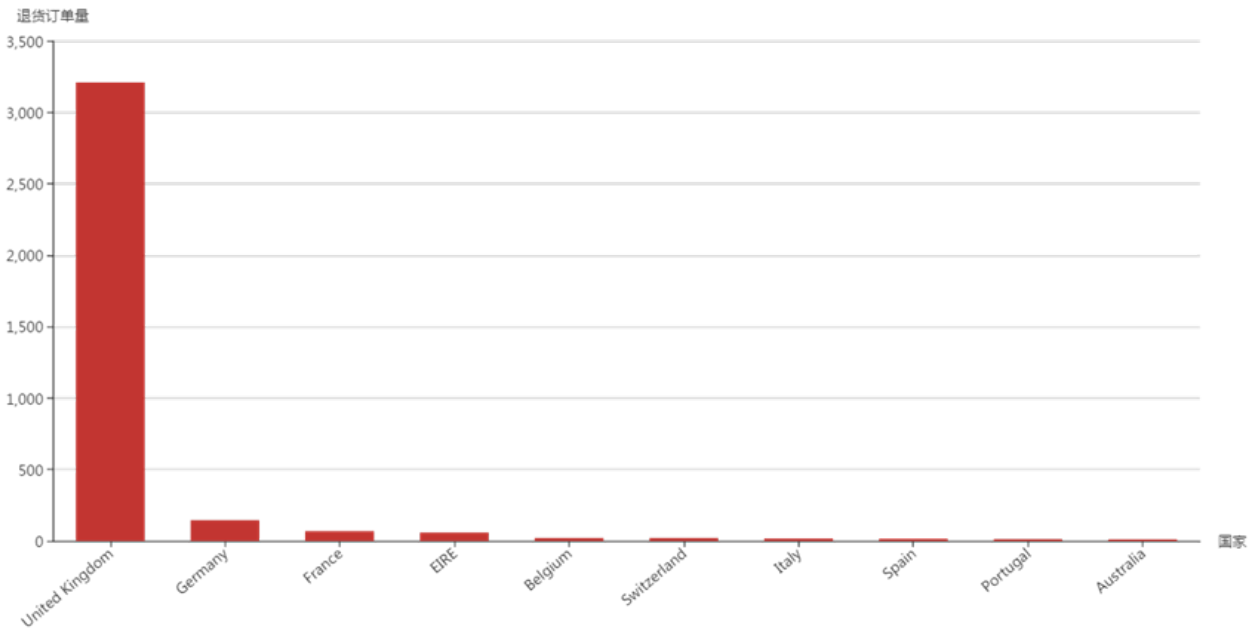
（4）销量最高的10个商品

——编号为84077的商品销量最高，达到53215件；销量Top3的商品在数量上差距并不大。



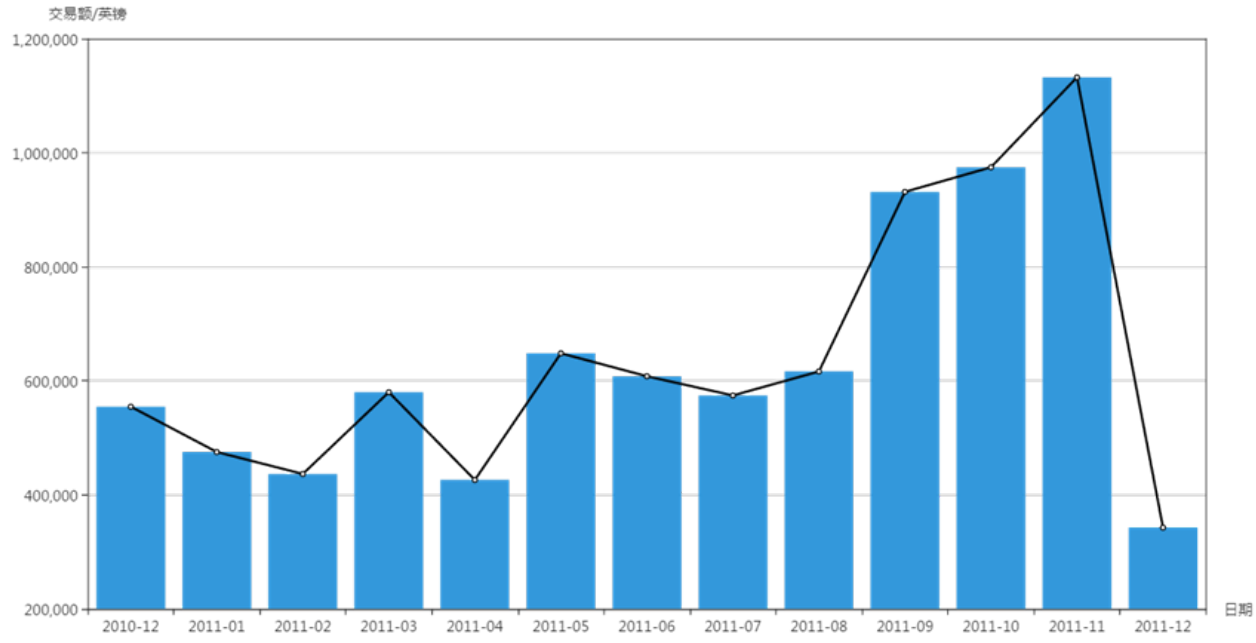
（5）商品描述的热门关键词Top300

——热门关键词包括bag、red、heart、pink、christmas、cake等。



（6）退货订单数最多的10个国家

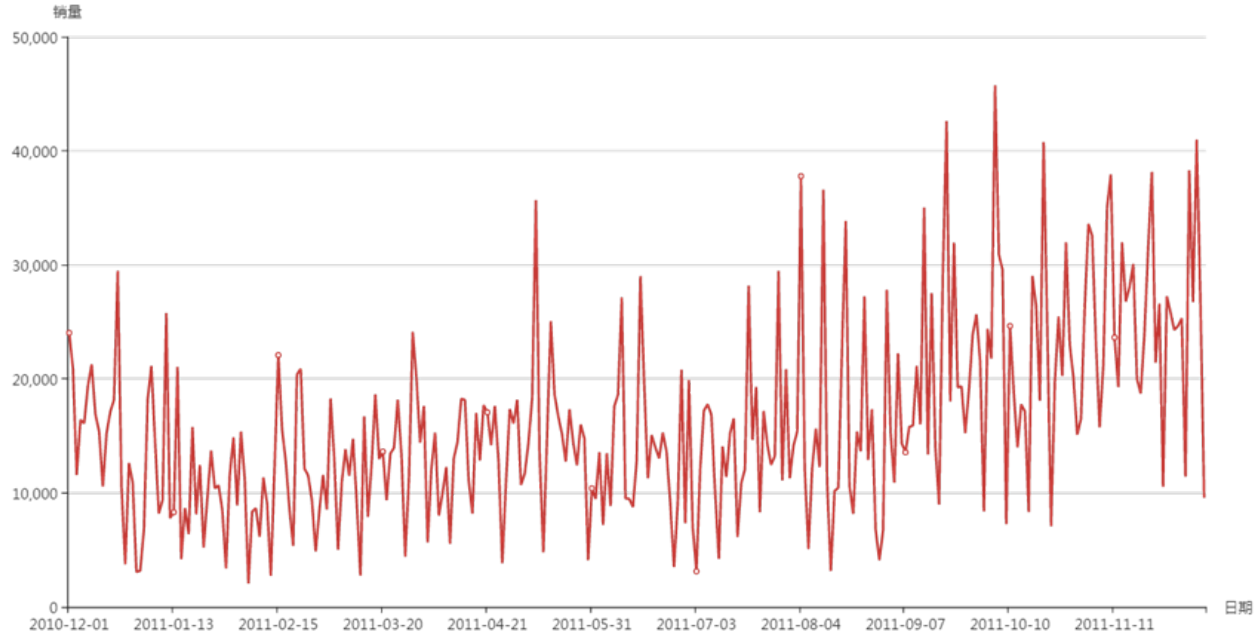
——英国的退货订单最多，达到3208个，远大于其他国家；其次是德国、法国、爱尔兰等。



2.关系

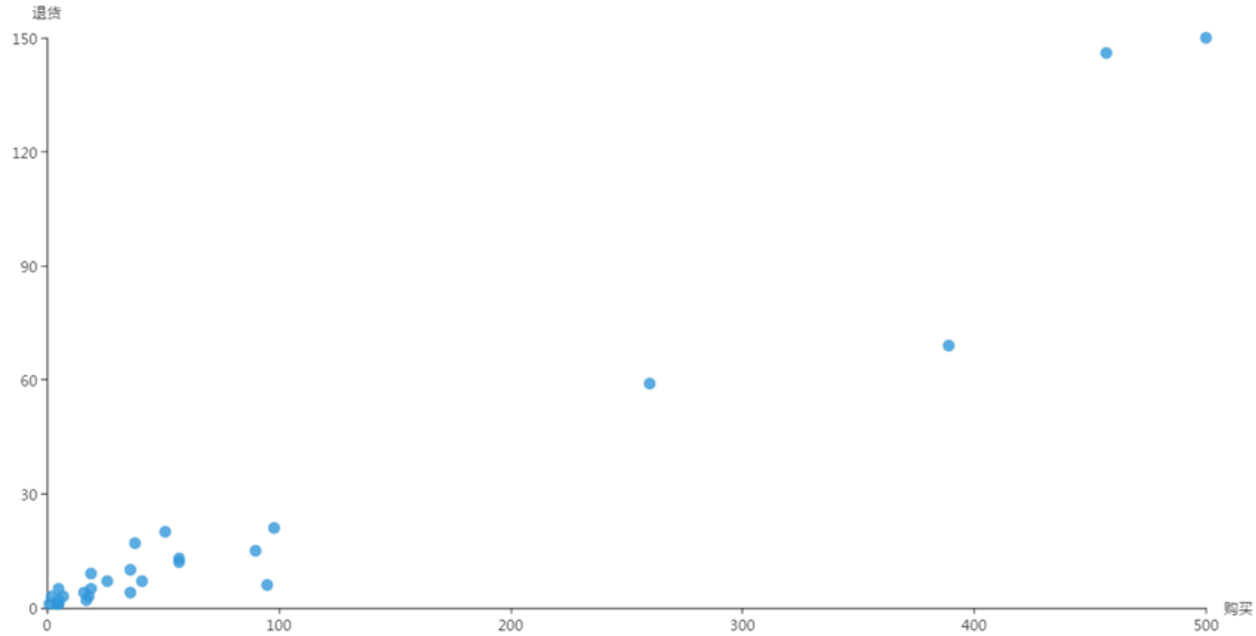
（7）月销售额随时间的变化趋势

——销售额较高的月份主要集中在下半年；由于该公司主要售卖礼品，并且下半年的节日较多，所以销售额比上半年高；2011年12月的销售额较低是因为数据只统计到2011/12/9 。



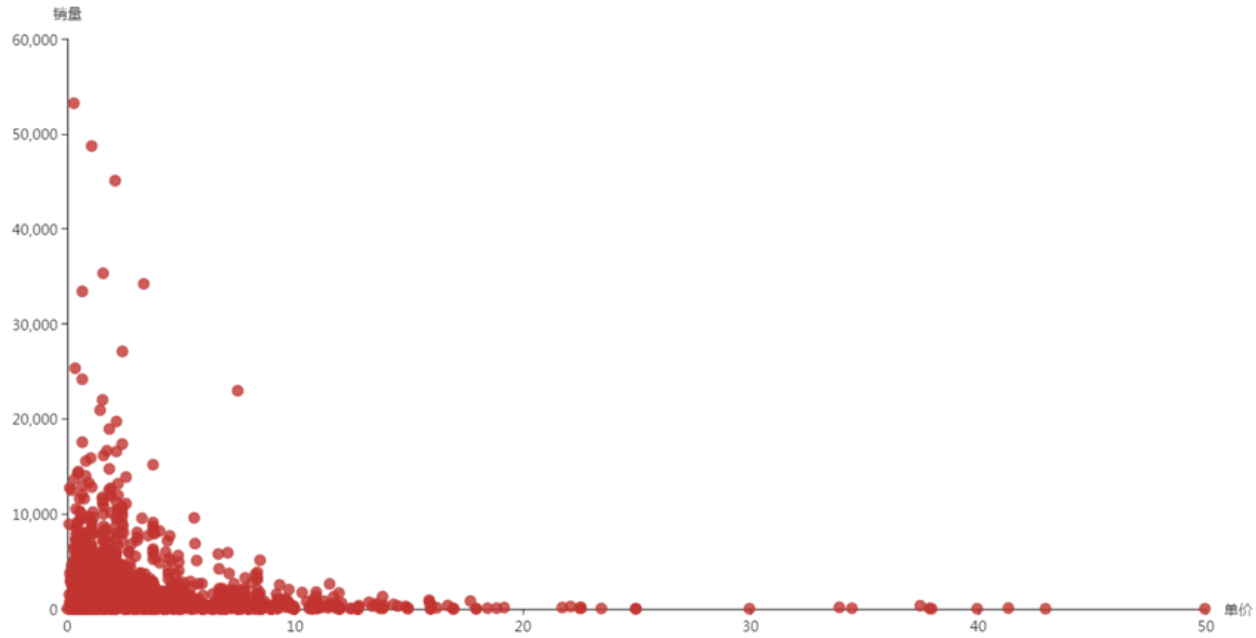
（8）日销量随时间的变化趋势

——下半年的日销量整体上高于上半年；2011年10月5号达到日销量的最高纪录45741件。



（9）各国的购买订单量和退货订单量的关系

——购买订单量越大的国家，退货订单量往往也越大。



（10）商品的平均单价与销量的关系

——总体上看，商品的销量随着平均单价的升高而下降。

六、总结

综合应用了多个技术，包括利用分布式文件系统HDFS读写数据集；使用pyspark交互式编程环境对数据进行预处理；利用python语言编写应用程序，使用Spark Core的RDD编程和Spark SQL编程实现对数据的统计分析；最后使用Bottle启动web服务，结合Echarts可视化框架完成统计结果的展示。